Papelería Matrix: plan de pruebas

Qué se prueba, con qué herramienta, qué resultado se espera, qué se ha probado ya y qué falta.

Objetivo y alcance

Comprobar que cada cambio que entra a `main` funciona y no rompe lo que ya estaba. Este plan cubre lo que existe hoy: la base de datos, la API (servicios, productos, ropa, errores y seguridad básica) y la página pública. Lo que todavía no existe, como el login, las facturas o las ventas, queda en la sección de pendientes.

Las pruebas se corren en **modo demo**, con una base H2 en memoria y datos de ejemplo, para que no dependan de XAMPP. El modo local se comprueba aparte, a mano.

Niveles de prueba

NIVEL	HERRAMIENTA	QUÉ CUBRE	CÓMO SE CORRE
Automáticas del backend	JUnit 5 y MockMvc	24 pruebas de la API: rutas, validaciones, errores, clave y CORS.	Dentro de `backend` : `mvn verify`
Colección de la API	Postman	15 peticiones con sus verificaciones, contra una API encendida.	Importar la carpeta `postman` , elegir el entorno y pulsar Run
Integración continua	GitHub Actions	Compilar y probar el backend, y construir y arrancar la imagen de Docker.	Automática en cada PR y en cada commit a `main`
Manuales	Navegador, XAMPP	Scripts SQL, modo local y la página pública.	Casos de las secciones 5 y 6

Pruebas automáticas del backend

Son 24 pruebas en 6 clases, todas en `backend/src/test/java/com/papeleriamatrix` .

ID	CLASE	QUÉ COMPRUEBA	REQUISITO
PA-01	`PapeleriaMatrixApplicationTests`	La aplicación arranca con toda su configuración.	Base
PA-02	`EstadoControllerTest`	Con el perfil demo activo, `/api/estado` informa el modo «demo».	Base
PA-03		Sin perfil activo, informa el modo «local».	Base

PA-04	`ServicioControllerTest`	Lista los servicios de ejemplo.	RF-06, CU-04
PA-05		No permite crear sin la clave de la API.	RF-03
PA-06		No permite crear con una clave incorrecta.	RF-03
PA-07		Crea, consulta, actualiza y elimina un servicio, y después responde 404 al consultarlo.	RF-03 a RF-06 CU-01 a CU-05
PA-08		Rechaza un código de servicio repetido con un 409.	Regla de negocio
PA-09		Rechaza datos inválidos y dice qué campos fallaron.	RF-42
PA-10		Rechaza un cuerpo que no es JSON válido.	RF-42, RNF-05
PA-11		Responde 404 con un mensaje claro si el servicio no existe.	RNF-05
PA-12	`CatalogoControllerTest`	Los productos no muestran el precio de compra, el lugar de compra ni la cantidad.	RF-01 Privacidad del negocio
PA-13		Consulta un producto por su id.	RF-01, CU-10
PA-14		Lista las 4 prendas de ejemplo.	RF-23, CU-40
PA-15		Responde 404 si el producto o la prenda no existen.	RNF-05
PA-16		Responde 400 con un mensaje claro si el id no es un número.	RNF-05, RF-42
PA-17	`RutasTest`	La raíz redirige a la documentación.	Usabilidad
PA-18		Una ruta que no existe responde 404, no 500.	RNF-05
PA-19		El `favicon.ico` que pide el navegador no genera un error del servidor.	RNF-05
PA-20		Un método que la ruta no admite responde 405.	RNF-05
PA-21	`CorsConfigTest`	Permite consultar desde el frontend local (puerto 5500).	Seguridad
PA-22		Responde la consulta previa que hace el navegador antes de un POST.	Seguridad
PA-23		Rechaza peticiones de otros sitios.	Seguridad
PA-24		El error 401 por falta de clave también lleva los permisos CORS, para que el navegador lo vea.	Seguridad

Resultado de hoy: 24 de 24 pasan, tanto en el equipo de desarrollo como en el CI de GitHub sobre el commit `962e95e`.

Colección de Postman

La carpeta `postman` trae la colección y dos entornos: Local (`http://localhost:8080` , clave `clave-de-prueba`) y Render (la clave se copia desde el panel de Render). Se corre con la API encendida en modo demo, porque espera los datos de ejemplo.

CARPETA	PETICIONES	QUÉ VERIFICA
1. Estado	2	La API está en línea y dice su modo. `/actuator/health` responde UP.
2. Servicios	7	Listar, crear, consultar, actualizar, crear con código repetido (409), eliminar y consultar el eliminado (404).
3. Seguridad y errores	2	Crear sin clave (401) y crear con datos inválidos (400, con los campos que fallaron).
4. Catálogos públicos	4	Listar y consultar productos y ropa. El producto no trae el precio de compra.

Resultado de hoy: las 15 peticiones se reprodujeron con un script equivalente contra la API en modo demo y dieron 15 de 15 bien. La sintaxis de los 16 scripts de verificación se revisó con Node. **Falta pulsar Run en la aplicación de Postman**; no se ha hecho todavía.

Integración continua

El archivo `.github/workflows/ci.yml` corre en cada pull request hacia `main` y en cada commit que llega a `main` . Tiene dos trabajos, y Render solo despliega si los dos pasan.

TRABAJO	QUÉ HACE	SE CONSIDERA APROBADO SI
Backend (compilar y probar)	Java 21 y `mvn -B verify` dentro de `backend` .	Compila y pasan las 24 pruebas.
Docker (construir la imagen)	Construye el `Dockerfile` , arranca el contenedor y consulta `/actuator/health` hasta 60 segundos.	La imagen se construye y la API responde.

Pruebas manuales hechas

Hechas el 30 de octubre de 2026 por quien desarrolló el cambio. Resultado esperado y obtenido coinciden en todos los casos.

ID	CASO	RESULTADO ESPERADO	ESTADO
PM-01	Ejecutar los 30 scripts de `database/scripts` en orden, en MariaDB 10.4 (la de XAMPP).	15 tablas y 14 relaciones, sin errores.	• **Pasó**
PM-02	Arrancar la API en modo local contra esa base.	Arranca y `ddl-auto: validate` no encuentra diferencias entre entidades y tablas.	• **Pasó** Con una advertencia, ver pendientes.

PM-03	En modo local, crear un servicio con tilde por `POST /api/servicios` y mirar la tabla.	La fila queda guardada con la tilde bien.	• **Pasó**
PM-04	Página con la API encendida, a 1280 y a 375 píxeles de ancho.	Servicios, papelería y ropa salen de la API, con foto y precio. Sin desbordes laterales.	• **Pasó**
PM-05	Página con la API apagada.	Papelería y ropa muestran el catálogo de respaldo con un aviso. Servicios muestra un mensaje y el botón Reintentar.	• **Pasó**
PM-06	Pulsar Reintentar con la API ya encendida.	Los servicios cargan y el botón desaparece.	• **Pasó**
PM-07	Buscadores de cada sección: «sobre», «CARTÓN carta», «zzz», y vacío.	2 resultados, 1 resultado, mensaje de sin resultados y lista completa.	• **Pasó**
PM-08	Buscador general: «horario sábados», «precio de copias», «telefono», «blusa m».	Cada búsqueda encuentra lo que corresponde, aunque falten las tildes.	• **Pasó**
PM-09	Menú de celular: abrir, cerrar con Escape y cerrar al elegir un enlace.	El panel ocupa la pantalla y el botón cambia su estado. Cerrado, no se puede recorrer con el teclado.	• **Pasó**
PM-10	Aviso de privacidad y términos y condiciones.	Aparece en la primera visita, se puede aceptar y no vuelve a salir.	• **Pasó**
PM-11	Formulario de registro y de login con datos vacíos, inválidos y completos.	Mensajes claros. Ninguno da acceso, y lo dicen.	• **Pasó**
PM-12	Construir la imagen de Docker y arrancar el contenedor.	La API responde `UP`.	• **Pasó** En el CI de GitHub.

Lo que todavía no se ha probado

- **La colección en la aplicación de Postman.** Solo se probó con un script equivalente.
- **Modo local con MariaDB 10.4.** Funciona, pero Hibernate 7 pide oficialmente MariaDB 10.6 o MySQL 8 y muestra una advertencia `HHH000511` porque MariaDB se anuncia como «5.5.5». Conviene actualizar XAMPP a la versión 10.11.
- **Render.** El servicio todavía no está creado. No se ha probado el despliegue ni el arranque en frío, que tarda hasta un minuto en el plan gratuito.
- **La página en un teléfono real.** Solo se probó en el navegador integrado, con el ancho de pantalla emulado.
- **RNF-09, 2,5 segundos.** No se ha medido el tiempo de carga.
- **RNF-02, accesibilidad.** Hay medidas básicas, pero no una auditoría con lector de pantalla ni con una herramienta como Lighthouse.
- **RNF-11, cientos de inserciones por tabla.** No hay prueba de volumen.
- **Funciones que aún no existen.** Login, roles, sesión de 15 minutos, operadores, facturas, ventas, ingresos, estadísticas y reportes.

Pruebas que debe traer cada tarea nueva

Cada PR de código incluye sus pruebas, copiando el patrón de `ServicioControllerTest`. Como mínimo:

TAREA	PRUEBAS MÍNIMAS
T1 y T2 Productos y ropa	Crear, consultar, actualizar y eliminar. Sin clave devuelve 401. Datos inválidos devuelve 400. Código repetido devuelve 409. La respuesta pública sigue sin mostrar los datos internos.
T3 Usuario y Administrador	La contraseña se guarda con hash y nunca se devuelve. Nombre de usuario, documento y correo repetidos devuelven 409.
T4 Información del negocio	Consultar es público. Escribir pide clave. La información queda asociada a un administrador existente.
T5 Operadores y claves	Un operador solo existe asociado a un usuario. Despedir cambia su estado sin borrar su historial. La clave de permiso se guarda con hash.
T6 Facturas de servicio	Cabecera y detalle se guardan juntos o no se guarda ninguno. El código de factura es único. Modificar o eliminar exige la clave de permiso.
T7 Ventas	Vender descuenta el stock. Vender más de lo que hay devuelve un error claro. Modificar o eliminar devuelve el stock.
T8 Ingresos y reportes	El cálculo coincide con las facturas y ventas de prueba. El historial no se pierde al generar un reporte nuevo.

Criterios para aceptar un cambio

1. `mvn verify` pasa en el equipo de quien lo hizo.
2. Las dos revisiones del CI, «Backend» y «Docker», salen en verde en el PR.
3. Si cambia la API, la colección de Postman se actualiza y corre completa en modo demo.
4. Si cambia la página, se revisa a 1280 y a 375 píxeles de ancho.
5. Otra persona del equipo revisa el PR antes de mergearlo.

Cómo ejecutar todo

```
cd backend
mvn verify                                              # las 24 pruebas automáticas

mvn spring-boot:run "-Dspring-boot.run.profiles=demo"   # API en modo demo, puerto 8080
                                                        # luego, en Postman: Run con el
entorno Local
```

El CI corre solo al abrir el PR. Su resultado aparece abajo en la página del PR, en la sección de revisiones.